Synopsis

Problem Statement

Project Id: AIML-19

Name : Member1 : Rounak Tayal

Class Roll Number: 56

University Roll Number :2028826

Section and Group: U and (G2)

Name : Vansh Choudhary

Class Roll Number: 65

University Roll Number : 2028898

Section and Group: U and (G2)

Name : Keshav Sharma

Class Roll Number: 45

University Roll Number : 2028741

Section and Group: U and (G2)

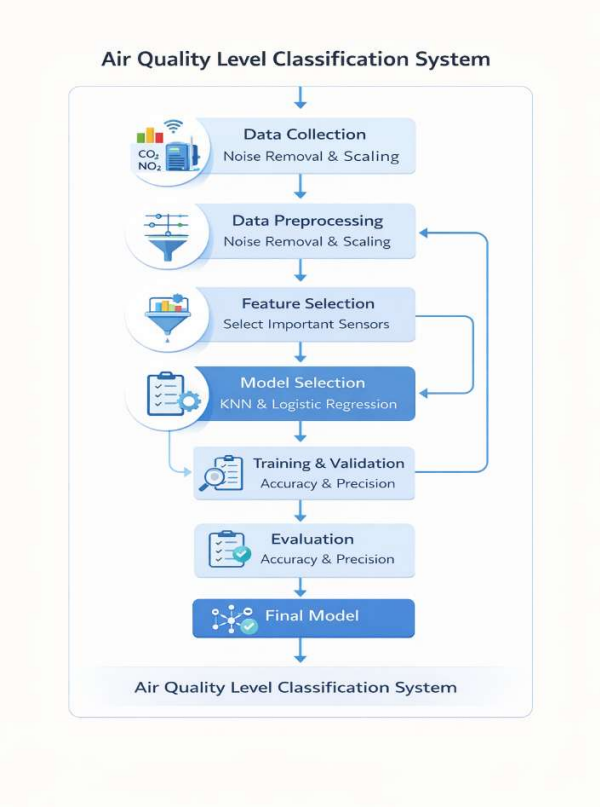
1. Problem Description

Air pollution is one of the most serious environmental challenges being faced today. Monitoring and predicting air quality accurately is important for public health, urban planning, and environmental protection. Manual monitoring is slow, costly, and not always reliable. The Air Quality Prediction System addresses this problem by using machine learning to automatically classify air quality as GOOD, Moderate, or WORST based on environmental sensor readings such as CO, NO2, Temperature, and Humidity.

In real life, this system can help government agencies issue timely alerts to citizens, help industries reduce harmful emissions, and assist researchers in studying pollution patterns. The system automates data preprocessing, feature selection, and model training to produce accurate predictions with minimal manual effort, making air quality monitoring faster and more scalable.

2. Steps of Implementation

- Load the dataset (Air_Quality_noisy.csv) into a Pandas DataFrame
- Explore the dataset — check structure, data types, and statistical summary using df.info() and df.describe()
- Handle missing values — fill null entries using column mean (df.fillna(df.mean()))
- Remove duplicate records from the dataset
- Detect outliers using Boxplot visualization before handling
- Handle outliers using IQR method — clip values beyond Q1 − 1.5×IQR and Q3 + 1.5×IQR for columns CO, NO2, Temperature, Humidity
- Create the target column Air_Quality_Category by labeling AQI values as GOOD (≤ 50), Moderate (≤ 100), or WORST (> 100)
- Separate input features (X) and target variable (y)
- Split dataset into 80% training and 20% testing sets
- Apply Standard Scaler to normalize feature values
- Apply Forward and Backward Sequential Feature Selection to select top 3 relevant features
- Train machine learning models and evaluate performance using metrics



3. Proposed Modules

- Data Loading Module – Reads the CSV dataset and loads it into a Pandas DataFrame for further processing.
- Data Preprocessing Module – Handles missing values, removes duplicates, and clips outliers to clean the dataset before training.
- Feature Engineering Module – Creates the target label column (Air_Quality_Category) from raw AQI values.
- Feature Selection Module – Applies Forward and Backward Sequential Feature Selection to identify the most relevant input features.
- Model Training Module – Trains classification models on the processed and selected features using GridSearchCV for hyperparameter tuning.
- Evaluation Module – Computes accuracy, precision, recall, and F1-score to compare model performance.
- Visualization Module – Generates correlation heatmaps, boxplots, confusion matrices, and accuracy comparison charts.

4. Required Topics from the Subject

- Data Preprocessing – Techniques such as mean imputation for missing values, duplicate removal, and IQR-based outlier clipping are used to prepare clean data for model training.
- Feature Selection – Sequential Feature Selection (forward and backward) is applied to reduce dimensionality and select only the most informative features, which improves model accuracy.
- Classification Algorithms – Supervised learning algorithms like KNN, Logistic Regression, Decision Tree, Random Forest, and SVM are used to classify air quality into predefined categories.
- Model Evaluation Metrics – Metrics like accuracy, precision, recall, and F1-score are used to measure and compare how well each model performs on unseen test data.
- Data Visualization – Libraries like matplotlib and seaborn are used to plot boxplots, heatmaps, and confusion matrices to understand data distribution and model behavior.

5. Platform Required

- Visual Studio Code – For writing, editing, and debugging Python code. It provides useful extensions and an integrated terminal for development.
- Jupyter Notebook – For running the project interactively cell by cell and viewing outputs and visualizations inline.
- Python 3.x – Programming language used for the entire implementation.

6. Link Sources

- GeeksforGeeks – https://www.geeksforgeeks.org/
- Scikit-Learn Documentation – https://scikit-learn.org/
- TutorialsPoint – https://www.tutorialspoint.com/
- Stack Overflow – https://stackoverflow.com